Amortised analysis

CS3230 · Lecture 6

Demo course material, written for OpenPapr

Why amortise?

- Some operations are expensive, but rarely
- Worst case per operation is too pessimistic for a whole sequence

Amortised cost

- The average cost per operation over a worst-case sequence of operations
- Not an average over random inputs: no probability is involved

Aggregate method

- Bound the total cost T(n) of n operations
- Amortised cost = T(n)/n

Binary counter

- Bit i flips once every 2^i increments
- Total flips over n increments < 2n
- Amortised O(1) per increment

Stack with multipop

- Each element is pushed once and popped at most once
- n operations cost O(n) in total
- O(1) amortised per operation

Accounting method

- Charge each operation an amortised cost
- Store the extra as credit on the data structure
- Expensive operations spend the credit

Credit invariant

- Credit must never go negative
- Prove it holds after every operation

Potential method

- Phi maps each state of the data structure to a number
- Amortised cost = actual cost + change in Phi
- Phi starts at 0 and never goes negative

Example: dynamic array

- Double the capacity when the array is full
- Copying costs n, but only after about n cheap appends

Dynamic array, accounting

- Charge 3 per append
- 1 pays for the append, 2 are saved for future copies
- O(1) amortised per append

Summary

- Aggregate, accounting, potential
- All three give the same bound when done right